



Laboratorio di Programmazione

Lezione 9 – Allocazione Dinamica Memoria, File

Marco Anisetti (teoria)

Dipartimento di Informatica
marco.anisetti@unimi.it

Matteo Luperto (lab. turno A)

Dipartimento di Informatica
matteo.luperto@unimi.it

Nicola Bena (lab. turno B)

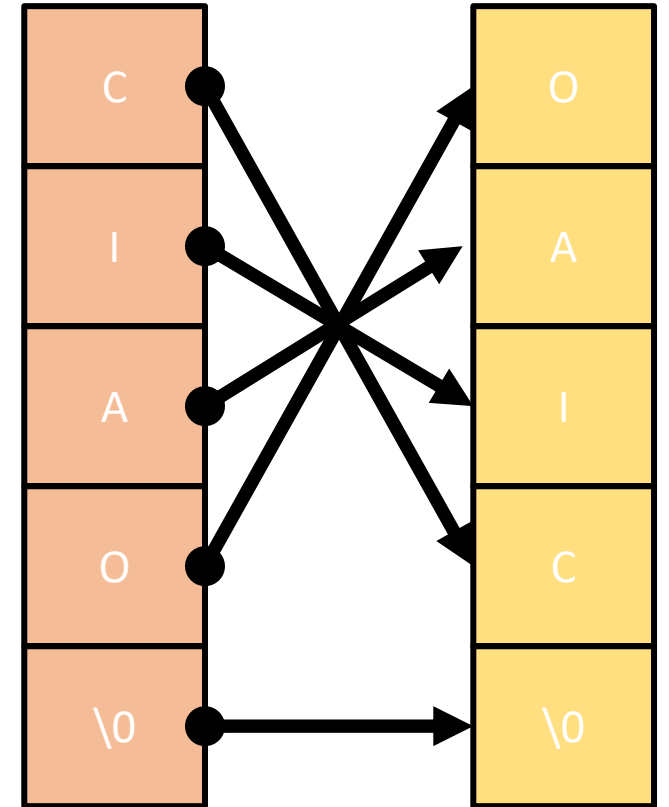
Dipartimento di Informatica
nicola.bena@unimi.it

Trova l'Errore

Scrivete un programma che chieda all'utente di inserire una stringa, invochi una funzione che restituisce una copia della stringa scritta in ordine inverso, la visualizzi e termini. La funzione che inverte la stringa ha prototipo

```
void reverse(char *s, char *t)
```

dove il primo parametro rappresenta la stringa da invertire, e il secondo la stringa invertita. In `main`, invocate la funzione passandole come primo parametro (il nome di un) array di tipo `char` inizializzato con una costante stringa — ossia la stringa letta dall'utente — e come secondo parametro (il nome di un) array di tipo `char` della medesima dimensione del primo. Un esempio di come potrebbe essere scritta la funzione `main` del programma è in Fig. 1.

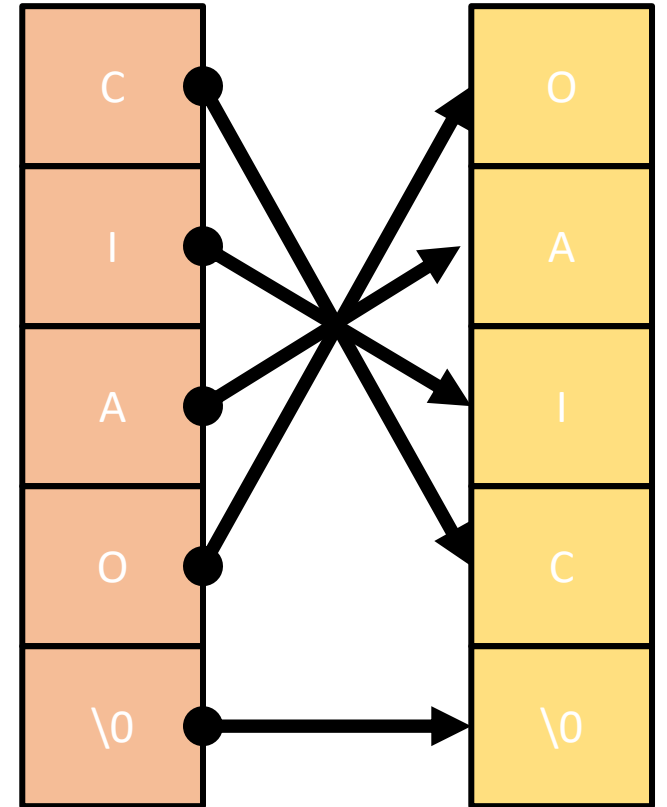


Trova l'Errore

```
void reverseEasy(char *s, char* srev)
{
    int length, c;
    char *end = s;
    length = strlen(s);

    for (c = 0; c < length - 1; c++)
        end++;

    for (c = 0; c < length; c++)
    {
        *srev = *end;
        srev++;
        end--;
    }
}
```



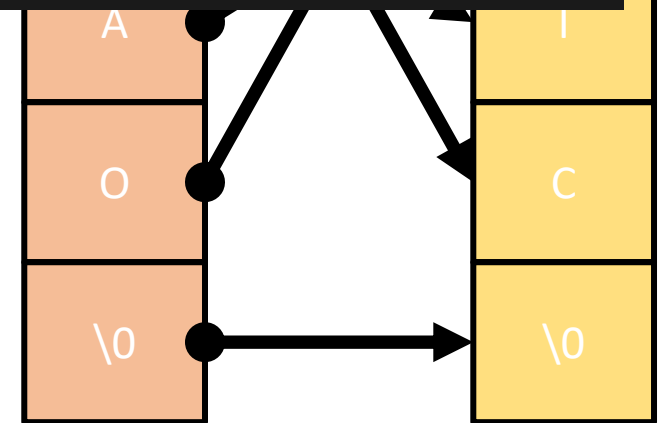
Esercizio 1 Lezione 8: reverse

```
void reverseEasy(char *s, char* srev)
{
    int length, c;
    char *end = s;
    length = strlen(s);

    for (c = 0; c < length - 1; c++)
        end++;

    for (c = 0; c < length; c++)
    {
        *srev = *end;
        printf("%d: %d\n", end-s, *end);
        srev++;
        end--;
    }
}
```

```
goldleaf@GoldBookPro2 L6 % ./a.out
GALILEO
6: 79
5: 69
4: 76
3: 73
2: 76
1: 65
0: 71
OELILAG
goldleaf@GoldBookPro2 L6 %
```



Stampo gli elementi su cui opero ad ogni step per debug

Esercizio 1 Lezione 8: reverse - main

Un main di test ci permette di testare se il programma funziona...

```
int main(void)
{
char str1[BUFSIZ] = "GALILEO";
char str2[BUFSIZ];

printf("%s\n", str1);

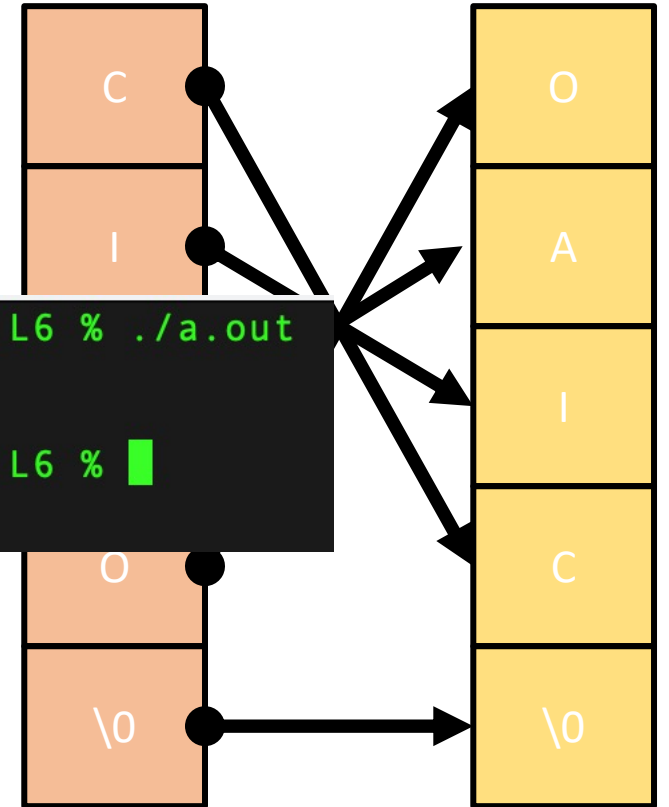
reverseEasy(str1, str2);

printf("%s\n", str2);

return 0;
}
```



```
[goldleaf@GoldBookPro2 L6 % ./a.out
GALILEO
OELILAG
goldleaf@GoldBookPro2 L6 % █
```



... ma siamo sicuri che questo sia sufficiente?

Esercizio 1 Lezione 8: reverse - main

Proviamo a cambiare il caso di test.

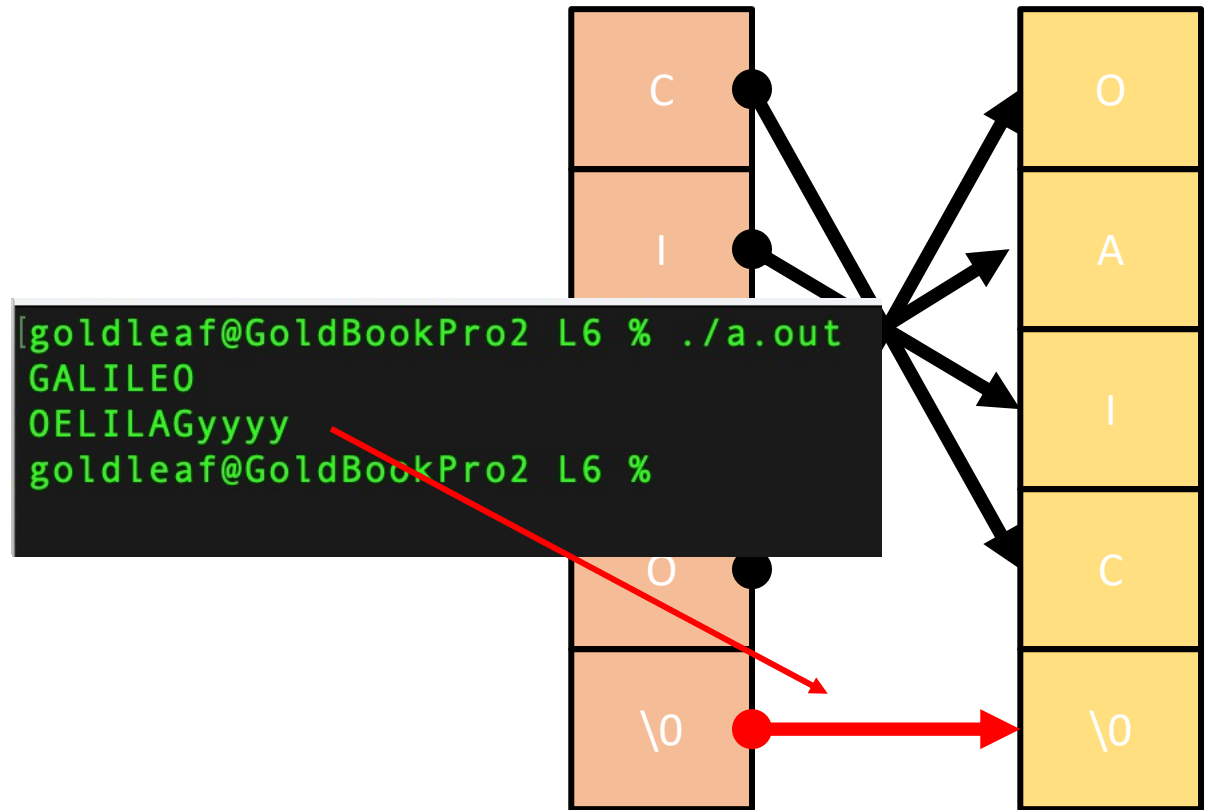
```
int main(void)
{
char str1[BUFSIZ] = "GALILEO\0xxxxx";
char str2[BUFSIZ] = "yyyyyyyyyyyy\0";

printf("%s\n", str1);

reverseEasy(str1, str2);

printf("%s\n", str2);

return 0;
}
```



Abbiamo trovato un errore, ma perché il programma funzionava?

Esercizio 1 Lezione 8: reverse - main

Proviamo a cambiare il caso di test.

```
int main(void)
{
char str1[BUFSIZ] = "GALILEO\0xxxxx";
char str2[BUFSIZ] = "yyyyyyyyyyyy\0";

printf("%s\n", str1);

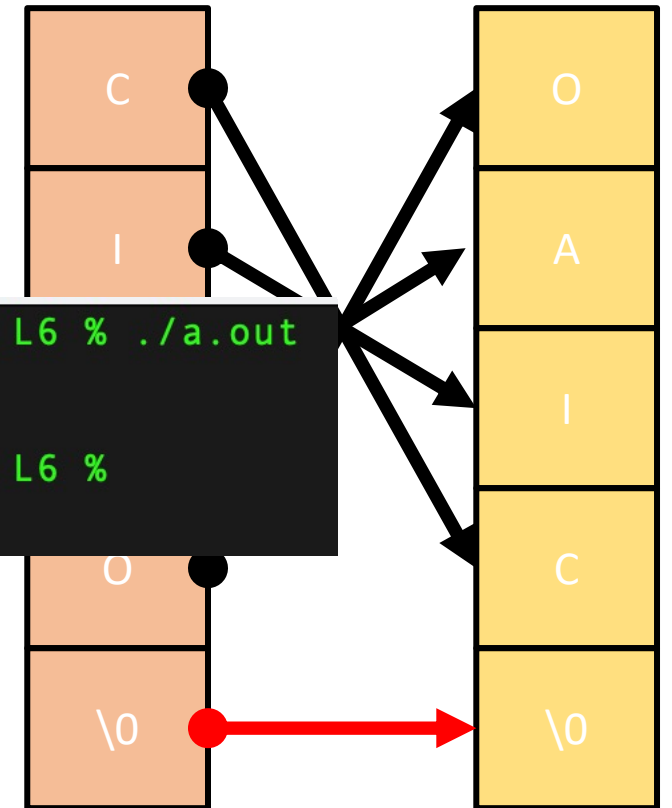
reverseEasy(str1, str2);

printf("%s\n", str2);

return 0;
}
```



```
[goldleaf@GoldBookPro2 L6 % ./a.out
GALILEO
OELILAGyyyy
goldleaf@GoldBookPro2 L6 %
```



Abbiamo trovato un errore, ma perché il programma funzionava?

Dopo la lettera 'G' c'era in memoria (per caso) uno 'O', in ASCII un '\0'

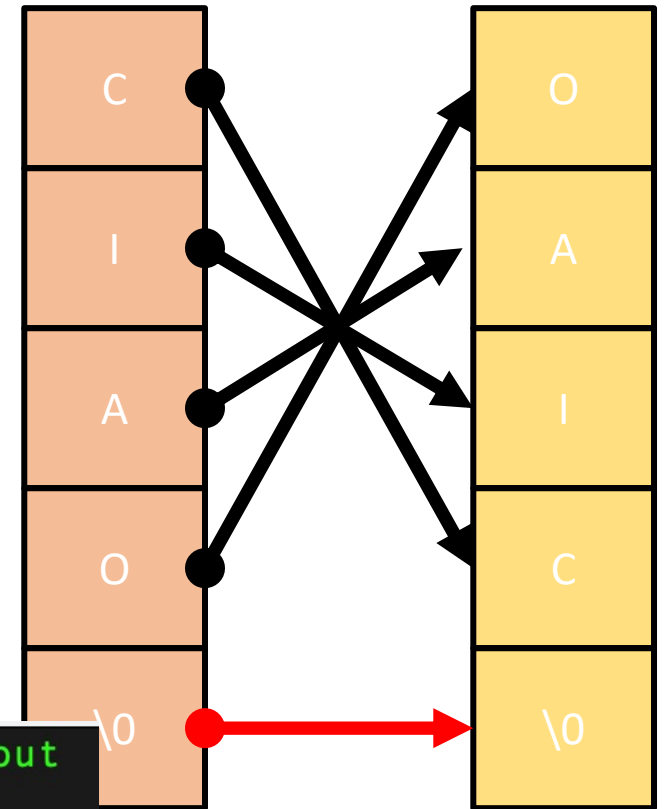
Esercizio 1 Lezione 8: reverse - bugfix

```
void reverseEasy(char *s, char* srev)
{
    int length, c;
    char *end = s;
    length = strlen(s);

    for (c = 0; c < length - 1; c++)
        end++;

    for (c = 0; c < length; c++)
    {
        *srev = *end;
        //printf("%d: %d\n", end-s, *end); per debug
        srev++;
        end--;
    }

    *srev = '\0';
}
```



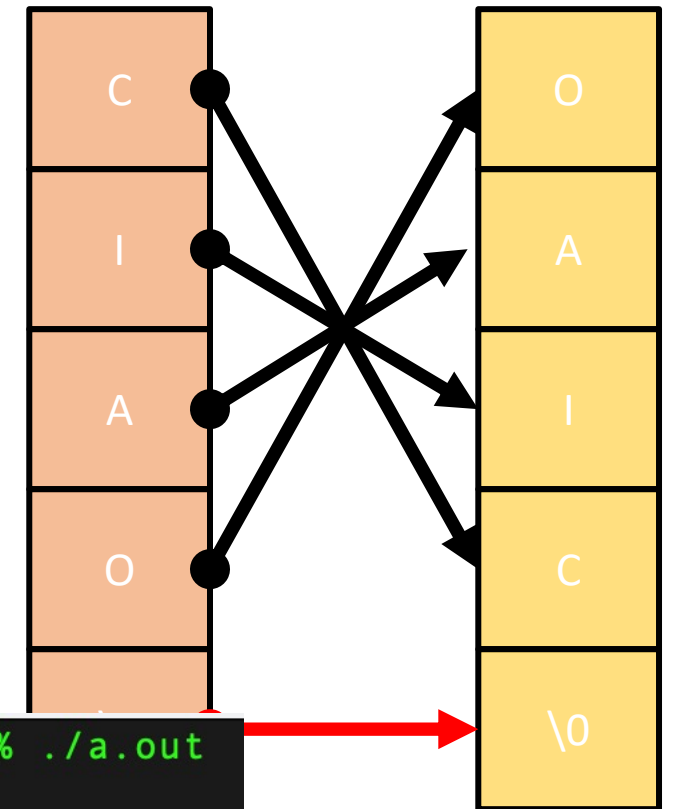
```
[goldleaf@GoldBookPro2 L6 % ./a.out
GALILEO
OELILAG
goldleaf@GoldBookPro2 L6 %
```

Esercizio 1 Lezione 8: reverse - bugfix

```
void reverseEasy(char *s, char* srev)
{
    int length, c;
    char *end = s;
    length = strlen(s);
    //for (c = 0; c < length - 1; c++)
    // end++;
    end += length-1; ←

    for (c = 0; c < length; c++)
    {
        *srev = *end;
        //printf("%d: %d\n", end-s, *end); per debug
        srev++;
        end--;
    }

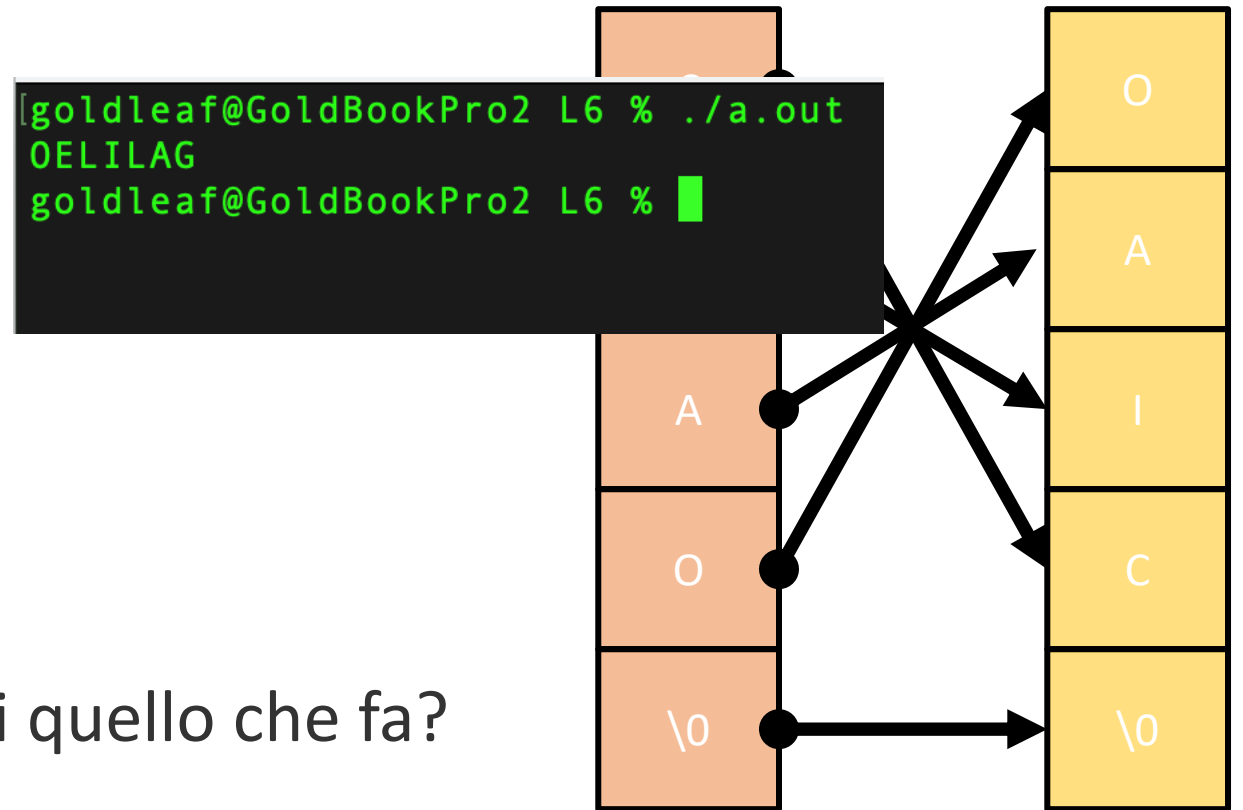
    *srev = '\\0';
}
```



```
[goldleaf@GoldBookPro2 L6 % ./a.out
GALILEO
OELILAG
goldleaf@GoldBookPro2 L6 %
```

Esercizio 1 Lezione 8: reverse – altri errori

```
void reverse (char * s, char * t){  
    int i = 0, j = 0;  
    int cont = 0;  
  
    while(*(s+i) != '\0') i++;  
  
    while((t[j] = s[i - 1])){  
        j ++;  
        i --;  
        cont ++;  
    }  
}
```

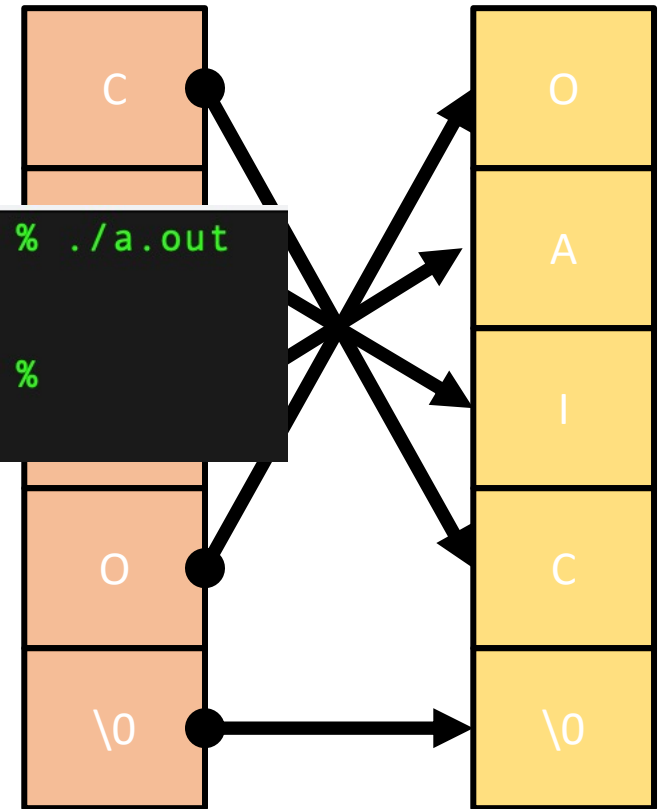


Sembra funzionare, ma siamo sicuri di quello che fa?

Esercizio 1 Lezione 8: reverse – altri errori

```
int main() {  
    char str1[BUFSIZ] = "GALILEO\0";  
    char str2[BUFSIZ] = "yyyyyyyyyy\0";  
    char s[BUFSIZ] = "\0?GALILEO\0xxxxx";  
    char t[BUFSIZ] = "yyyyyyyyyyyyyy\0";  
  
    reverse(str1, str2);  
    printf("%s\n", str2);  
  
    reverse(s+2, t);  
    printf("%s\n", t);  
    return 0;  
}
```

```
[goldleaf@GoldBookPro2 L6 % ./a.out  
OELILAG  
OELILAG?  
goldleaf@GoldBookPro2 L6 %
```



Nel secondo caso, non funziona, perché?

Esercizio 1 Lezione 8: reverse – altri errori

```
char str1[BUFSIZ] = "GALILEO\0";
char str2[BUFSIZ] = "yyyyyyyyyy\0";
char s[BUFSIZ] = "\0?GALILEO\0xxxxx";
char t[BUFSIZ] = "yyyyyyyyyyyyyy\0";
reverse(s+2, t);
[...]
```

```
void reverse (char * s, char * t){
    int i = 0, j = 0;
    int cont = 0;

    while(*(s+i) != '\0') i++;

    while((t[j] = s[i - 1])){
        j ++;
        i --;
        cont ++;
    }
    printf("%d %d %d\n", t[j], i-1, j);
}
```

Nel secondo caso, non funziona, perché?

```
[goldleaf@GoldBookPro2 L6 % ./a.out
0 -1 7
OELILAG
0 -2 8
OELILAG?
goldleaf@GoldBookPro2 L6 %
```

Esercizio 1 Lezione 8: reverse – altri errori

```
char str1[BUFSIZ] = "GALILEO\0";
char str2[BUFSIZ] = "yyyyyyyyyy\0";
char s[BUFSIZ] = "\0?GALILEO\0xxxxx";
char t[BUFSIZ] = "yyyyyyyyyyyyyy\0";
reverse(s+2, t);
[...]
```

```
void reverse (char * s, char * t){
    int i = 0, j = 0;
    int cont = 0;

    while(*(s+i) != '\0') i++;

    while((t[j] = s[i - 1])){
        j ++;
        i --;
        cont ++;
    }
    printf("%d %d %d\n", t[j], i-1, j);
}
```

```
goldleaf@GoldBookPro2 L6 % ./a.out
0 -1 7
OELILAG
0 -2 8
OELILAG?
goldleaf@GoldBookPro2 L6 %
```

-1	0	1	2	3	4	5	6	7
?	G	A	L	I	L	E	O	\0

- La funzione termina quando trova il primo 0 a destra della stringa s
- Questo perché (char)\0 è (int)0
- Risultato: la funzione può sembrare corretta, ma in realtà sta accedendo a dell'area di memoria proibita alla sinistra (prima) del primo array.
- Nel primo caso (chiamata su str1) funziona perché a destra di str1 c'è uno 0 (casualmente)
- Ovviamente questo è un **errore**

Esercizio 1 Lezione 8: reverse – altri errori

```
void reverseERRORE (char * s, char * t){  
    int i = 0, j = 0;  
    int cont = 0;  
  
    while(*(s+i) != '\0') i++;  
  
    while((t[j] = s[i - 1])){  
        j ++;  
        i --;  
        cont ++;  
    }  
}
```

```
void reverseFIX (char * s, char * t){  
    int i = 0, j = 0;  
    int cont = 0;  
  
    while(*(s+i) != '\0') i++;  
  
    *(t+i) = '\0';  
    while(i>0){  
        t[j] = s[i - 1];  
        j ++;  
        i --;  
        cont ++;  
    }  
}
```

Come sistemo?

- Cambio la condizione del while
- Aggiungo il terminatore

Esercizio 1 Lezione 8: reverse – altri errori

```
void reverseFIX2 (char * s, char * t){
    int i = 0, j;

    while(*(s+i) != '\0') i++;

    for(j=0; i>0; j++,i--)
        t[j] = s[i - 1];

    t[j]='\0';
}
```

```
void reverseFIX (char * s, char * t){
    int i = 0, j = 0;
    int cont = 0;

    while(*(s+i) != '\0') i++;

    *(t+i) = '\0';
    while(i>0){
        t[j] = s[i - 1];
        j ++;
        i --;
        cont ++;
    }
}
```



Meglio però riscrivere la stessa soluzione con un ciclo for, è più corretto

Esercizio 1 Lezione 8: reverse – altre soluzioni

```
void reverseINC(char *s, char *t){  
    int i,j;  
    for(i=0,j=strlen(s)-1;s[i];i++,j--)  
        t[j]=s[i];  
    t[i]='\0';  
}
```

- Questa soluzione copia prima le lettere a destra della stringa di source (s) nella stringa di destinazione. Quindi prima copia “G”, poi “A”, ...
- Termina quando trova il terminatore in S
- Infine aggiunge il terminatore

```
void reverseIDX(char *s, char *t)  
{  
    int length = strlen(s);  
    char *end;  
  
    for(int i=0; i<length;i++)  
        t[length-1-i] = s[i]  
    t[length]='\0';  
}
```

- Questa soluzione copia prima le lettere a sinistra della stringa di source (s) nella stringa di destinazione. Quindi prima copia “O”, poi “E”, ...
- Termina quando copia length elementi
- Infine aggiunge il terminatore.

Esercizio 1 Lezione 8: reverse – altre soluzioni

```
void reversePTR(char *s, char *t)
{
    int length = strlen(s);
    char *end;

    for(end = s + length - 1; end >= s; t++, end--)
        *t = *end;

    *t = '\0';
}
```

- Copia la prima le lettere a sinistra della stringa di source (s) nella stringa di destinazione.
Quindi prima copia “O”, poi “E”, ...
Per farlo usa direttamente due puntatori ai due array.
- Termina quando il puntatore che sta scorrendo il primo array (end), è arrivato al primo elemento dell’array stesso (puntato da s)
- Infine aggiunge il terminatore.

Array e Variable-Length Array

```
/* non usare VLA */  
int n;  
scanf("%d", &n);  
int a[n];  
for(i=0; i<n; i++)  
    scanf("%d", &a[i]);
```

```
/* usare le define*/  
#define N 100 // >> max size  
...  
int n, a[N];  
do {  
    scanf("%d", &n);  
} while (n>N);  
  
for(i=0; i<n; i++)  
    scanf("%d", &a[i]);
```

Invece di usare i VLA, sovradimensionate la dimensione degli array con una define, e usate i primi $n < N$ elementi (come con le stringhe con `BUFSIZ`).

Allocazione Dinamica di Memoria

C consente di allocare dinamicamente porzioni di memoria usabili durante l'esecuzione del programma

- Dinamicamente = allocare memoria la cui dimensione non è nota a tempo di compilazione
- La memoria richiesta viene allocata nello heap
- Funzioni disponibili in `stdlib.h`
- Allocano `n` byte contigui e ritornano un puntatore alla prima cella
- Grazie all'equivalenza array – puntatori, il puntatore può essere usato per scorrere la porzione allocata come un array
 - In generale necessarie quando serve riservare della memoria che non è nota a tempo di compilazione (prossime lezioni)

Funzione malloc

```
void * malloc(size_t size)
```

- Alloca `size` byte contigui e ritorna un puntatore alla prima cella
- `void *` è un tipo puntatore usato per riferirsi a zone di memoria con tipo arbitrario (casting necessario)
- L'allocazione può fallire, quindi occorre controllare che il valore ritornato sia un puntatore valido
- Se fallisce, il puntatore è uguale alla costante `NULL`

```
#include <stdlib.h>
```

```
int main(){
```

```
    int * array = (int*) malloc(10 * sizeof(int));
```

```
    if (array == NULL) {
```

```
        // allocazione fallita
```

```
    }
```

```
}
```

Funzione malloc

```
#include <stdlib.h>
#include <stdio.h>
int main(){
    int i;
    int * array;
    array = (int*) malloc(10 * sizeof(int));
    if (array == NULL){
        printf("malloc error\n");
        return -1;
    }
    for (i = 0; i < 10; i++)
        array[i] = i * i;
    for (i = 0; i < 10; i++)
        printf("array[%d]=%d\n", i, array[i]);
    return 0;
}
```

Funzione malloc

```
#include <stdlib.h>
#include <stdio.h>
int main(){
    int i;
    int * array;
    array = (int*) malloc(10 * sizeof(int));
    if (array == NULL){
        printf("malloc error\n");
        return -1;
    }
    for (i = 0; i < 10; i++)
        array[i] = i * i;
    for (i = 0; i < 10; i++)
        printf("array[%d]=%d\n", i, array[i]);
    return 0;
}
```

malloc prende in ingresso il numero di byte da allocare, quindi per simulare un array specifichiamo

- Numero di elementi da allocare *
- Dimensione in byte di un singolo elemento

Funzione malloc

- La porzione di memoria allocata è libera e disponibile esclusivamente per il programma, ma non è inizializzata (= non sappiamo cosa contiene)
- Assicurarsi di inizializzare prima di leggere

```
#include <stdlib.h>
#include <stdio.h>
int main(){
    int i;
    int * array;
    array = (int*) malloc(10 * sizeof(int));
    if (array == NULL){
        printf("malloc error\n");
        return -1;
    }
    for (i = 0; i < 10; i++)
        printf("array[%d]=%d\n", i, array[i]);
    return 0;
}
```

```
> ./es02_malloc_dirty
array[0]=0
array[1]=0
array[2]=0
array[3]=0
array[4]=0
array[5]=0
array[6]=0
array[7]=0
array[8]=0
array[9]=0
```

Funzione malloc

Come per ogni accesso tramite puntatore, non c'è controllo che la memoria acceduta sia valida e accessibile

```
#include <stdlib.h>
#include <stdio.h>

int main(){

int i;
int * array = (int*) malloc(10 *
    sizeof(int));

for (i = 0; i < 10000000; i++)
    printf("array[%d]=%d\n", i,
        array[i]);

return 0;
}
```

```
array[1030331]=0
array[1030332]=0
array[1030333]=0
array[1030334]=0
array[1030335]=0
array[1030336]=0
array[1030337]=0
array[1030338]=0
array[1030339]=0
array[1030340]=0
array[1030341]=0
array[1030342]=0
array[1030343]=0
array[1030344]=0
array[1030345]=0
array[1030346]=0
array[1030347]=0
fish: Job 1, './es03_malloc_boundary' terminated by signal SIGBUS (Misaligned address error)
```

Funzione calloc

```
void * calloc(size_t n, size_t size)
```

- Alloca `n` celle contigue di dimensione `size` byte, azzerandole prima
- L'allocazione può fallire, quindi occorre controllare che il valore restituito sia un puntatore valido
- Se fallisce, il puntatore è uguale alla costante `NULL`

```
#include <stdlib.h>

int main(){

    int * array = (int*) calloc(10, sizeof(int));
    if (array == NULL) {
        // allocazione fallita
    }
}
```

Funzione free

La memoria allocata dinamicamente dev'essere liberata quando non più necessaria, per evitare memory leakage

- `void free(void *ptr)`
 - Prende in input il puntatore alla prima cella allocata da `malloc/calloc` e libera l'intera porzione di memoria

```
#include <stdlib.h>

int main(){

    int * array = (int*) malloc(10 * sizeof(int));
    if (array == NULL) {
        // allocazione fallita
    }
    // uso array...
    free(array); // quando array non serve piu'
}
```

Funzione `free`

- La memoria allocata dinamicamente dev'essere liberata quando non più necessaria, per evitare memory leakage
- Tutti i puntatori che puntano alla porzione di memoria liberata non possono essere più essere usati, perché non contengono più indirizzi validi



Funzione `free`

- La memoria allocata dinamicamente dev'essere liberata quando non più necessaria, per evitare memory leakage
- Tutti i puntatori che puntano alla porzione di memoria liberata non possono essere più essere usati, perché non contengono più indirizzi validi



```
#include <stdlib.h>
#include <stdio.h>
```

```
int main(){
    int * array = (int*) malloc(10 * sizeof(int));
    free(array);
    printf("array[0]=%d\n", array[0]);
    return 0;
}
```

```
> ./es04_free_ub
array[0]=7
```

Matrici Dinamiche

- Modellata come un array dinamico di puntatori, ciascun puntatore punta a un array dinamico
- tipo: `int ** matrice`

```
#include <stdlib.h>

int main(){
    int **matrix;
    int m, n, i;

    matrix = (int**) calloc (m, sizeof(int *));
    for (i = 0; i < m; i++)
        matrix[i] = (int*) calloc(n, sizeof(int));

    // uso matrix, poi libero

    for (i=0; i < n; i++)
        free(matrix[i]);
    free(matrix);
}
```

Matrici Dinamiche

- Modellata come un array dinamico di puntatori, ciascun puntatore punta a un array dinamico
- tipo: `int ** matrice`

```
#include <stdlib.h>

int main(){
    int **matrix;
    int m, n, i;

    matrix = (int**) calloc (m, sizeof(int *));
    for (i = 0; i < m; i++)
        matrix[i] = (int*) calloc(n, sizeof(int));

    // uso matrix, poi libero

    for (i=0; i < n; i++)
        free(matrix[i]);
    free(matrix);
}
```

Errori da evitare con memoria dinamica

L'indirizzo di memoria di un dato allocato dinamicamente è noto solo a runtime ed è salvato in un puntatore, una variabile.

```
int a[N] = {1,2,3,4,5 6,7};  
int * array = (int*) malloc(10 * sizeof(int));  
  
...  
  
array = a; // perdo riferimento alla memoria puntata!
```

È quindi possibile perdere il riferimento alla memoria (per errore); la memoria rimarrà allocata e persa fino alla fine del programma, e sarà inaccessibile.

Questo tipo di errore è un *memory leak*

Memory leaks e funzioni

```
int myfunction(int n){  
    int * array = (int*) malloc(n * sizeof(int));  
    // codice della funzione  
    return 0; // perdo riferimento alla memoria puntata!  
}
```

Particolare attenzione va prestata ad allocazioni dinamiche fatte all'interno di funzioni:

- Il riferimento alla memoria allocata vive nel *record di attivazione* della funzione, che viene liberato al termine della funzione stessa
- La funzione deve o liberare la memoria prima di terminare (`free`), o restituire un puntatore alla memoria allocata al chiamante
- In caso contrario, *memory leak* → errore

Questo tipo di errori sono gravi, perché di fatto non influiscono sul risultato del vostro programma ma ne peggiorano significativamente le prestazioni.

Input/Output e File

- `stdio.h` definisce i tipi e le funzioni per gestire l'input e l'output
 - Abbiamo già visto `printf` e `scanf` & co
- Tipo `FILE` usato per interagire con i file
- Flusso standard
 1. “Aprire” il file usando `fopen`, che restituisce `*FILE`
 2. Leggere/scrivere su file usando `fprintf`, `fscanf`, `fgets` & co
 3. Chiudere il file usando `fclose`

Stream

- I file sono gestiti come stream (cioè flussi di caratteri/byte) indipendentemente dallo specifico dispositivo di input/output
- Stream standard: `stdin`, `stdout`, e `stderr`
 - Non occorre aprirli e chiuderli
 - Definiti in `stdio.h`
 - `printf`, `scanf` & co sono versioni specifiche di funzioni di I/O che operano sugli stream standard

Funzioni di Input/Output

- `FILE *fopen(char *nomefile, char *modo)`
- `int fclose(FILE *stream)`
- `int fscanf(FILE *stream, char * format, ...)`
- `int fprintf(FILE *stream, char *format, ...)`
- `char *fgets(char *s, int n, FILE * stream)`
- `int fflush(FILE *stream)`
- `int feof(FILE *stream)`

Funzioni di Input/Output

- `FILE *fopen(char *nomefile, char *modo)`
 - Restituisce un puntatore → controllare che non sia `NULL` 
 - `nomefile` è il percorso assoluto o relativo al file
 - `modo` identifica come il file dev'essere aperto
 - `r`: lettura
 - `w`: scrittura, posizionandosi all'inizio del file
 - `a`: scrittura, posizionando in coda al file ("append")
 - Se il file è binario (no testo puro) `rb`, `wb`, `ab`
 - Se il file non esiste e viene aperto con `w` o `a`, viene creato, vuoto, al percorso `nomefile`, altrimenti restituisce `NULL`
 - Se ci sono errori (ad es., il file non esiste), restituisce `NULL`

Funzioni di Input/Output

- `int fclose(FILE *stream)`
 - Chiude il file `stream*` dopo che è stato aperto
 - Libera la memoria allocata nello heap, usata per gestire le operazione di I/O
 - Restituisce 0 se ha avuto successo, EOF altrimenti

Funzioni di Input/Output

- `int fscanf(FILE *stream, char * format, ...)`
 - Analoga a `scanf` con l'eccezione del primo parametro: interpreta il contenuto di `stream` sulla base di `format`
 - Restituisce il numero di elementi assegnati
 - Restituisce EOF se al termine della lettura, arriva in fondo al file 
 - La funzione `int feof(FILE *stream)` restituisce vero (`>0`) se si è arrivati in fondo a `stream`

Funzioni di Input/Output

- `int fscanf(FILE *stream, char * format, ...)`
 - Analoga a `scanf` con l'eccezione del primo parametro: interpreta il contenuto di `stream` sulla base di `format`
 - Restituisce il numero di elementi assegnati
 - Restituisce `EOF` se, al termine della lettura, arriva in fondo al file 
 - La funzione `int feof(FILE *stream)` restituisce vero (`>0`) se si è arrivati in fondo a `stream`
- `char *fgets(char *s, int n, FILE *stream)`
 - Legge una riga fino al primo '`\n`' (incluso), fermandosi dopo `n-1` caratteri al più, assegnandola a `s*`
 - `n-1` perché l'`n`-esimo carattere è '`\0`'
 - Restituisce `s*`, oppure `NULL` se raggiunta la fine del file o in caso di errore

Funzioni di Input/Output

- `int fprintf(FILE *stream, char *format, ...)`
 - Analoga a `printf` con l'eccezione del primo parametro: scrive su `stream` sulla base di `format` il contenuto delle espressioni seguenti
 - Restituisce il numero di caratteri scritti, EOF o altri valori <0 in caso di errore
- `int fflush(FILE *stream)`
 - Forza lo svuotamento del buffer: le operazioni di I/O avvengono in memoria per performance e sono effettivamente scritte su file in maniera asincrona

Funzioni di Input/Output

- `int fscanf(FILE *stream, char *format, ...)`
 - `scanf(format, ...) → fscanf(stdin, format, ...)`
- `int fprintf(FILE *stream, char *format, ...)`
 - `printf(format, ...) → fprintf(stdout, format, ...)`

```
#include <stdio.h>
```

```
int main(){  
    int n1, n2;  
  
    fprintf(stdout, "Inserisci un numero (fprintf): ");  
    scanf("%d", &n1);  
    printf("Inserisci un numero (printf): ");  
    fscanf(stdin, "%d", &n2);  
  
    fprintf(stdout, "Hai inserito: %d e %d\n", n1, n2);  
    return 0;  
}
```

```
> ./io  
Inserisci un numero (fprintf): 10  
Inserisci un numero (printf): 11  
Hai inserito: 10 e 11
```

File: Esempio

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]){
    FILE * file_r, *file_w;
    char dst[BUFSIZ];

    // controllare argc/argv!
    file_r = fopen(argv[1], "r");
    file_w = fopen(argv[2], "w");
    if (file_r == NULL || file_w == NULL){
        fprintf(stderr, "open error");
        return -1;
    }
}
```

```
while(fgets(dst, BUFSIZ, file_r) != NULL){
    // rimuovo a capo (lezione 4)
    dst[strcspn(dst, "\n")] = '\0';
    fprintf(file_w, "Ho letto: %s\n", dst);
}
if (fclose(file_r) != 0){
    fprintf(stderr, "close error\n");
    return -1;
}
if (fclose(file_w) != 0){
    fprintf(stderr, "close error\n");
    return -1;
}
}
```

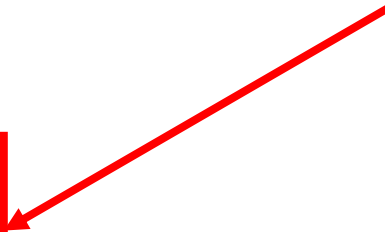
File: Esempio

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]){
    FILE * file_r, *file_w;
    char dst[BUFSIZ];

    // controllare argc/argv!
    file_r = fopen(argv[1], "r");
    file_w = fopen(argv[2], "w");
    if (file_r == NULL || file_w == NULL){
        fprintf(stderr, "open error");
        return -1;
    }
}
```

Apre file e controlla che l'apertura
sia andata a buon fine



File: Esempio

```
while(fgets(dst, BUFSIZ, file_r) != NULL){  
    // rimuovo a capo (lezione 4)  
    dst[strcspn(dst, "\n")] = '\0';  
    fprintf(file_w, "Ho letto: %s\n", dst);  
}  
if (fclose(file_r) != 0){  
    fprintf(stderr, "close error");  
    return -1;  
}  
if (fclose(file_w) != 0){  
    fprintf(stderr, "close error");  
    return -1;  
}  
}
```

1. esegue fgets leggendo una riga dal file e scrivendola in dst
2. Valuta se il valore restituito da fgets è diverso da NULL
 - È NULL in caso di end of file/errore
 - Quindi rimane nel ciclo finché la lettura va a buon fine (cioè finché il valore è diverso da NULL)

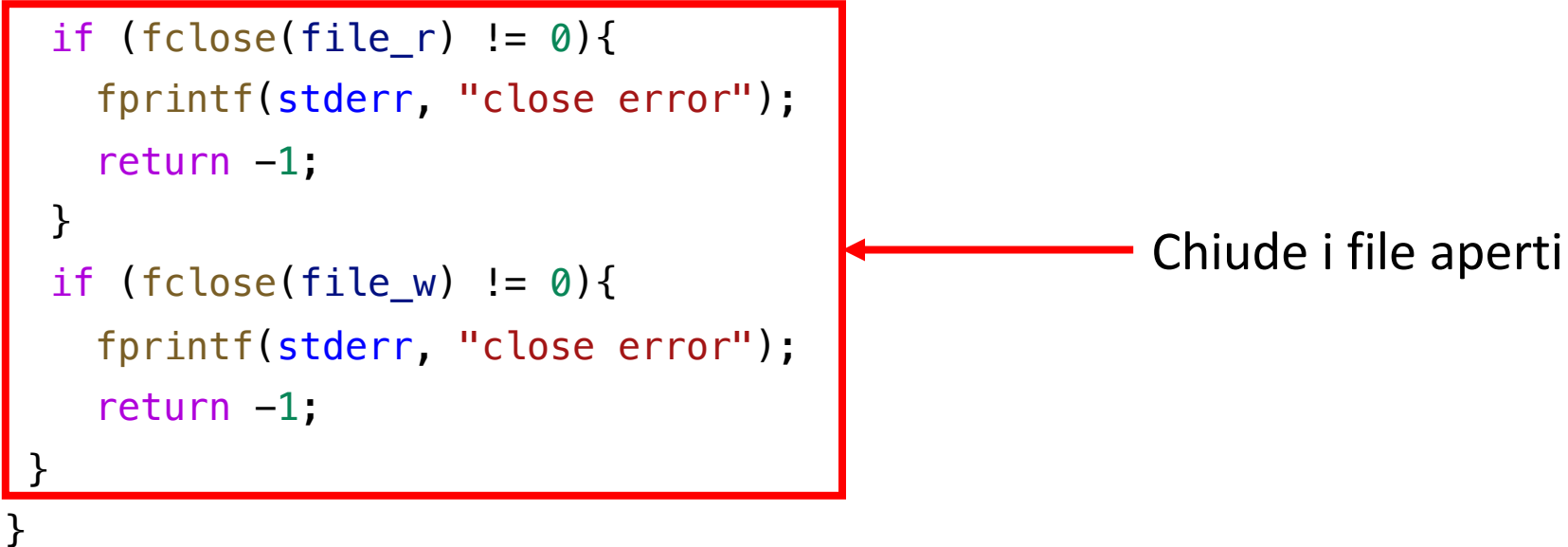
File: Esempio

```
while(fgets(dst, BUFSIZ, file_r) != NULL){  
    // rimuovo a capo (lezione 4)  
    dst[strcspn(dst, "\n")] = '\0';  
    fprintf(file_w, "Ho letto: %s\n", dst);  
}  
if (fclose(file_r) != 0){  
    fprintf(stderr, "close error");  
    return -1;  
}  
if (fclose(file_w) != 0){  
    fprintf(stderr, "close error");  
    return -1;  
}  
}
```

Scrive quanto letto, preceduto da "Ho
letto: " nel file destinazione

File: Esempio

```
while(fgets(dst, BUFSIZ, file_r) != NULL){  
    // rimuovo a capo (lezione 4)  
    dst[strcspn(dst, "\n")] = '\0';  
    fprintf(file_w, "Ho letto: %s\n", dst);  
}  
  
if (fclose(file_r) != 0){  
    fprintf(stderr, "close error");  
    return -1;  
}  
  
if (fclose(file_w) != 0){  
    fprintf(stderr, "close error");  
    return -1;  
}  
}
```



Chiude i file aperti